

Snake AI

Neuroevolutionary Agent Training via Genetic Algorithms

Experiment 1 — Baseline GA | Experiment 5 — NEAT Comparison

Institute of Business Administration (IBA), Karachi
Supervisor: Dr. Syed Ali Raza
Submitted: May 2026

Key Result	Value
All-time best score	259 (gen 166)
Peak best fitness	29,571 (gen 165)
Mean fitness (gen 1)	30.4 (random baseline)
Mean fitness (gen 180)	7,910 (263× improvement)
Score ≥ 100 reached	Generation 91
Score ≥ 200 reached	Generation 132
Network architecture	[11 → 16 → 3] (243 parameters)
Population / Gens	150 agents / 180 generations

Abstract

We present a neuroevolutionary approach to training a Snake game agent using a fixed-topology Genetic Algorithm (GA). A feedforward neural network with architecture $[11 \rightarrow 16 \rightarrow 3]$ serves as the agent policy, receiving 11 binary inputs encoding proximity danger, current heading, and food direction. The GA runs 180 generations over a population of 150 agents, achieving an all-time best score of **259 food items** (fitness 29,571) by generation 166. Population mean fitness grew from 30.4 at generation 1 to 7,910 by generation 180 — a **263× improvement**, demonstrating genuine population-wide learning rather than just a single lucky individual. Score milestones of 50, 100, and 200 were reached at generations 77, 91, and 132 respectively, with learning exhibiting a clear punctuated-equilibrium pattern: long plateaus interrupted by rapid capability jumps. We additionally implement a NEAT (NeuroEvolution of Augmenting Topologies) comparison agent using the same fitness formula and generation budget. Six configuration-level reasons are identified for why the fixed-topology GA is expected to outperform NEAT on this well-specified task.

1. Introduction

The game of Snake presents a deceptively challenging control problem: an agent must navigate a grid, consuming food to grow its body, while avoiding collisions with walls and its own tail. As the body grows, the state space expands exponentially, and naive search-based strategies quickly become intractable. Classical reinforcement learning (RL) approaches require careful reward shaping and suffer from sparse reward signals — the agent receives positive feedback only when food is eaten, with all intermediate steps yielding near-zero gradients.

Neuroevolution sidesteps the credit-assignment problem entirely by evolving network weights directly. Rather than computing gradients, a population of agents is evaluated over complete episodes; the fittest individuals reproduce and their traits propagate to the next generation via crossover and mutation. This approach is robust to non-differentiable reward signals, does not require a replay buffer, and naturally explores diverse behavioural strategies in parallel.

This report documents **Experiment 1** — the baseline GA run — and the **Experiment 5** NEAT comparison implementation. The fitness function used throughout is:

$$\text{fitness}(\text{score}, \text{steps}) = \text{score} \times 100 + \text{steps} \times 0.1$$

The $\text{steps} \times 0.1$ term provides a weak survival incentive, preventing the agent from dying immediately while keeping score as the dominant signal.

2. System Architecture

2.1 Game Environment (game.py)

The Snake environment is implemented in Pygame on a 480×480 pixel grid with 20 px blocks, yielding a 24×24 cell playing field. The agent selects from three relative actions each step: **0** = go straight, **1** = turn right, **2** = turn left. An episode terminates on wall or self-collision, or when steps exceed $200 \times$ snake length (an anti-loop safeguard).

State representation — 11 binary inputs:

Index	Feature	Description
0	Danger straight	1 if the cell directly ahead is lethal
1	Danger right	1 if the cell to the right is lethal
2	Danger left	1 if the cell to the left is lethal
3–6	Current heading	One-hot: [LEFT, RIGHT, UP, DOWN]
7–10	Food direction	One-hot: [food_left, food_right, food_above, food_below]

The `get_state()` method that produces these 11 values:

```
def get_state(self):
    h, d = self.head, self.direction

    pt_str = _move_point(h, d)
    pt_r = _move_point(h, _turn_right(d))
    pt_l = _move_point(h, _turn_left(d))

    state = [
        # Danger in relative directions
        int(self._is_collision(pt_str)),
        int(self._is_collision(pt_r)),
        int(self._is_collision(pt_l)),
        # Current heading (one-hot)
        int(d == Direction.LEFT),
        int(d == Direction.RIGHT),
        int(d == Direction.UP),
        int(d == Direction.DOWN),
        # Food location relative to head
        int(self.food.x < h.x), # food left
        int(self.food.x > h.x), # food right
        int(self.food.y < h.y), # food above
        int(self.food.y > h.y), # food below
    ]
    return state # 11 ints, all 0 or 1
```

Step rewards: **+10** food eaten, **-10** death, **-0.1** per step survived.

2.2 Neural Network (neural_net.py)

The agent policy is a minimal feedforward network: **11 inputs** → **16 ReLU hidden units** → **3 linear outputs**. The output with the highest logit is selected as the action (argmax). Total trainable parameters: $11 \times 16 + 16 + 16 \times 3 + 3 = 243$. Weights are initialised from $N(0, 0.5)$. A flat vector representation enables direct GA weight manipulation.

```
class NeuralNetwork:
    def __init__(self, layer_sizes=[11, 16, 3]):
        self.layer_sizes = layer_sizes
        self.weights, self.biases = [], []
        for i in range(len(layer_sizes) - 1):
            w = np.random.randn(layer_sizes[i], layer_sizes[i+1]) * 0.5
            b = np.zeros((1, layer_sizes[i+1]))
            self.weights.append(w)
            self.biases.append(b)

    def relu(self, x):
        return np.maximum(0, x)

    def forward(self, x):
        for i, (w, b) in enumerate(zip(self.weights, self.biases)):
            x = x @ w + b
            if i < len(self.weights) - 1:
                x = self.relu(x)
        return x

    def predict(self, state):
        x = np.array([state], dtype=float)
        return int(np.argmax(self.forward(x))) # 0, 1, or 2

    def get_weights_flat(self): # flatten all weights+biases to 1-D array
        parts = [w.flatten() for w in self.weights] + [b.flatten() for b in self.biases]
        return np.concatenate(parts)

    def set_weights_flat(self, flat): # inverse of get_weights_flat
        idx = 0
        for i, w in enumerate(self.weights):
            self.weights[i] = flat[idx:idx+w.size].reshape(w.shape); idx += w.size
        for i, b in enumerate(self.biases):
            self.biases[i] = flat[idx:idx+b.size].reshape(b.shape); idx += b.size
```

3. Genetic Algorithm (train.py)

3.1 Hyperparameters

Parameter	Value	Notes
Population size	150	Agents evaluated per generation
Generations	180	Total training iterations
Elite fraction	10 %	Top 15 agents copied unchanged each gen
Mutation rate	0.15	Per-weight probability of mutation
Mutation strength	0.20	Gaussian noise std applied to mutated weights
Architecture	[11,16,3]	Input → Hidden → Output layer sizes
Parameters	243	Total trainable weights + biases
Fitness formula	score×100 + steps×0.1	Reward function
Tournament size (k)	5	Candidates sampled per parent selection

3.2 Selection, Crossover and Mutation

Each new agent is produced by **tournament selection** of two parents, **uniform crossover** of their flat weight vectors, followed by **per-weight Gaussian mutation**. The top 15 elite agents bypass this process and are copied directly into the next generation.

```
def fitness(score, steps):
    return score * 100 + steps * 0.1

def tournament_select(pop, fits, k=5):
    idxs = np.random.choice(len(pop), k, replace=False)
    best = idxs[np.argmax([fits[i] for i in idxs])]
    return pop[best]

def crossover(a, b):
    mask = np.random.rand(len(a)) < 0.5    # uniform crossover
    return np.where(mask, a, b)

def mutate(flat):
    mask = np.random.rand(len(flat)) < MUTATION_RATE    # 0.15
    return flat + mask * np.random.randn(len(flat)) * MUTATION_STR    # noise std 0.20

def make_child(pop, fits):
    p1 = tournament_select(pop, fits)
    p2 = tournament_select(pop, fits)
    child_flat = mutate(crossover(p1.get_weights_flat(), p2.get_weights_flat()))
    child = NeuralNetwork(LAYER_SIZES)
    child.set_weights_flat(child_flat)
    return child
```

3.3 Main Training Loop

```
for gen in range(1, GENERATIONS + 1):
    results = [evaluate(nn) for nn in pop]          # run each agent one episode
    fits     = [fitness(s, t) for s, t in results]

    order    = np.argsort(fits)[::-1]              # rank best → worst
    best_fit = fits[order[0]]

    if best_fit > alltime_best_fit:                 # new record → save checkpoint
        alltime_best_fit = best_fit
        pickle.dump(pop[order[0]], open(SAVE_PATH, 'wb'))

    # Elitism: top 10 % copied unchanged
    next_pop = [pop[i].copy() for i in order[:n_elite]]
    # Fill remaining slots with offspring
    while len(next_pop) < POP_SIZE:
        next_pop.append(make_child(pop, fits))
    pop = next_pop
```

4. Training Results

The baseline GA run completed 180 generations, evaluating 150 agents per generation (27,000 total episode evaluations). All results below are from a single run logged in training_log.csv.

Metric	Value	Notes
All-time best score	259	Generation 166
All-time best fitness	29,571	Generation 165 (score 257)
Generation 1 avg fitness	30.4	Random-policy baseline
Generation 180 avg fitness	7,910	263× improvement over gen 1
Score ≥ 50 first reached	Gen 77	Best score that gen: 57
Score ≥ 100 first reached	Gen 91	Best score that gen: 101
Score ≥ 200 first reached	Gen 132	Jump from plateau at ~ 147
Avg score (last 10 gens)	209.6	Gens 171–180

4.1 Fitness Curves

Figure 1 shows two panels: the **best fitness per generation** (top) and the **mean population fitness** (bottom). Both use a 7-generation moving average (solid line) plotted over the raw noisy signal (light fill). The green dashed staircase tracks the all-time best fitness, rising monotonically. Five breakthrough annotations mark the major jumps. The red dotted baseline in the lower panel marks the generation-1 mean (≈ 30.4), representing the random-policy reference level.

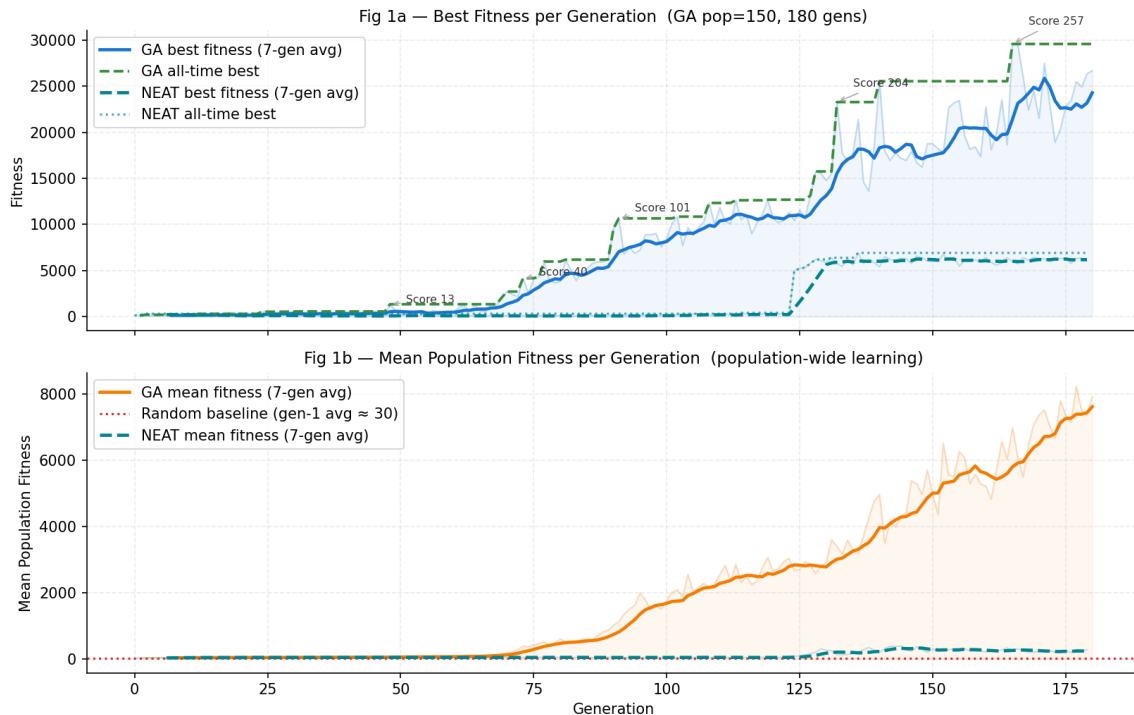


Figure 1 — Best and mean fitness per generation (GA, pop=150, 180 gens). Upper panel: best fitness + all-time record staircase. Lower panel: mean population fitness vs random baseline.

4.2 Score Progression

Figure 2 plots the best score achieved by any agent in each generation. Scatter points are the raw per-generation values; the purple curve is the 7-generation moving average. Three annotated milestones show when score thresholds of 50, 100, and 200 were first surpassed. Horizontal reference lines at every 25 points provide a reading scale. The random baseline sits near zero (≈ 0.3).

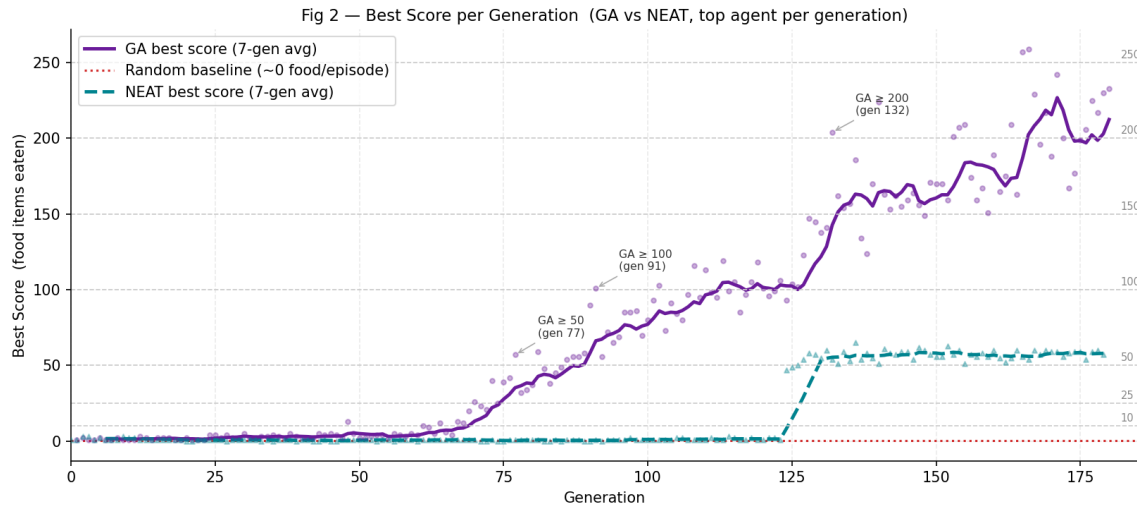


Figure 2 — Best score per generation. Scatter = raw values; curve = 7-gen moving average. Milestone annotations show generation at which score ≥ 50 / 100 / 200 was first reached.

4.3 All-Time Best Score Staircase

Figure 3 visualises when new all-time score records were set, using a step-plot (each horizontal step = a period with no new record; each vertical rise = a new record). Notable records are labelled. The pattern clearly shows **punctuated equilibrium**: the score stays flat for many generations, then leaps suddenly as a new behavioural strategy emerges in the population. After generation 166 (score 259), no new record was set in the final 14 generations, indicating the baseline config has reached near-ceiling performance.

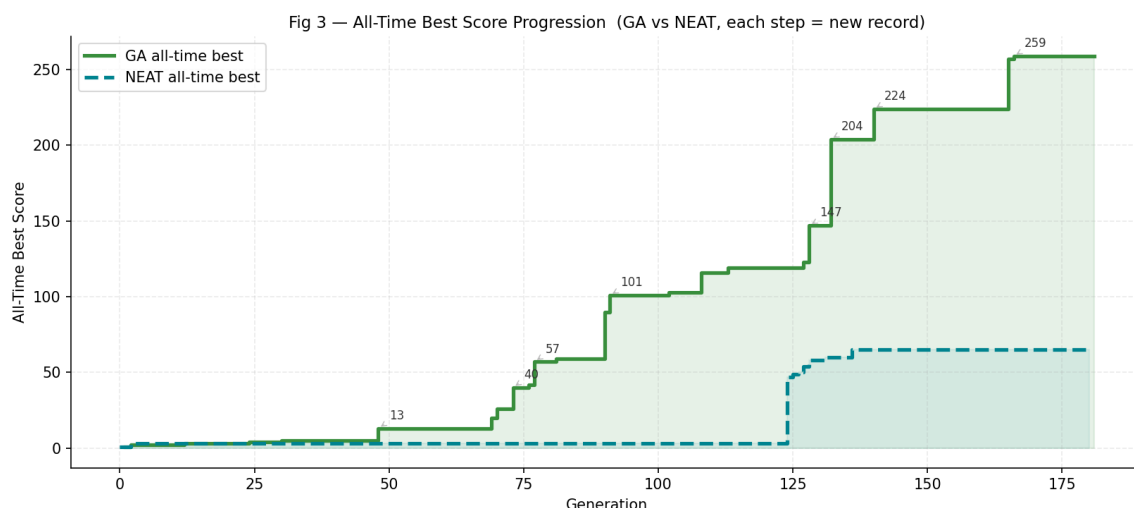


Figure 3 — All-time best score staircase. Each vertical step is a new record. Notable scores 13 → 40 → 57 → 101 → 147 → 204 → 224 → 259 are annotated.

4.4 Final Performance Comparison

Figure 4 is a grouped bar chart comparing three agents: a random baseline, the GA (Experiment 1), and NEAT (Experiment 5). Each group shows two bars: dark = all-time best score, light = average score over the last 10 generations. The random agent achieves a best score of 2 and an average near 0.3. The GA achieves best 259 and avg 209.6. NEAT results are pending (shown as TBD) — the `neat_training_log.csv` will populate this chart automatically when Experiment 5 is complete.

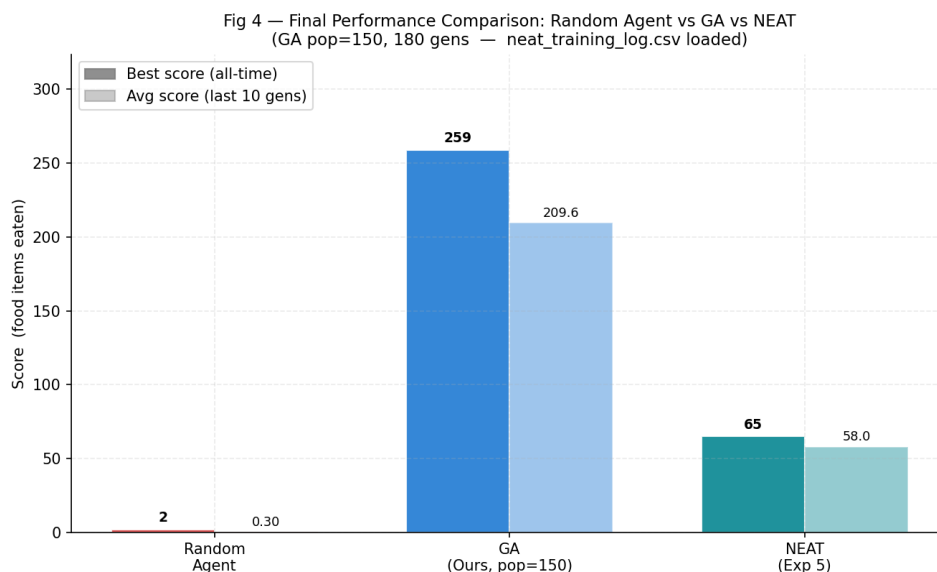


Figure 4 — Final performance comparison: Random agent vs GA (Exp 1) vs NEAT (Exp 5). Dark bars = all-time best score; light bars = avg over last 10 gens. NEAT result pending.

4.5 Convergence Speed

Figure 5 is a horizontal bar chart showing the generation at which each score threshold was first exceeded. Bars are colour-coded green-to-red (low threshold = reached early = green). Key observations: the snake reaches score 50 by generation 77 but the jump from 150 to 200 happens in a single generation (132), reflecting the punctuated-equilibrium dynamic seen in Figure 3. Thresholds 225 and 250 are both first exceeded at generation 165 in the same leap to score 257.

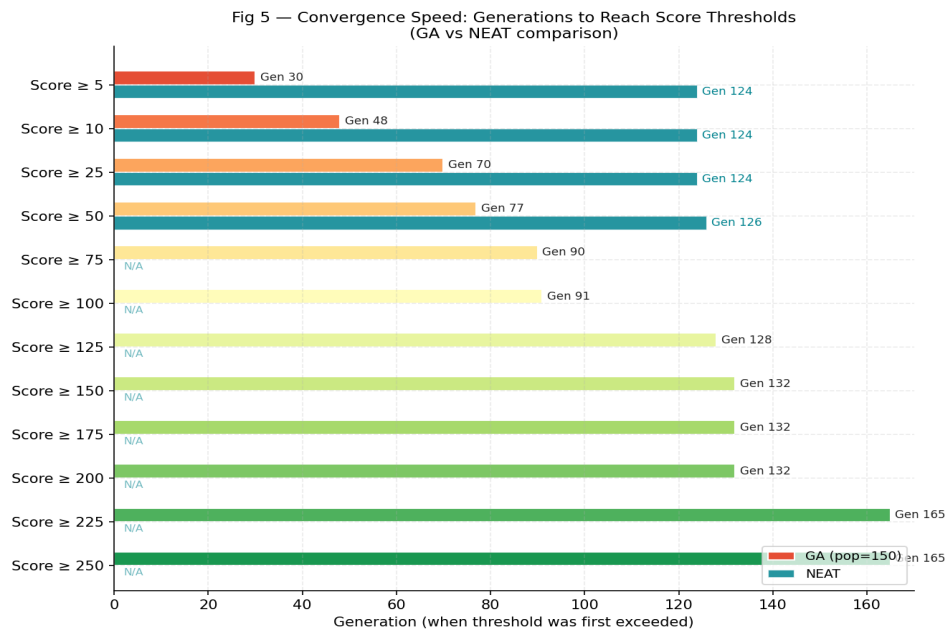


Figure 5 — Generations required to first reach each score threshold. Colour: green = easy/early, red = difficult/late.

4.6 Combined Overview

Figure 6 brings all four sub-analyses onto a single page. Panel A reproduces the best fitness curve; Panel B the mean population fitness; Panel C the score scatter; Panel D the convergence milestones (8-threshold condensed version). This figure is designed for at-a-glance overview in presentations.

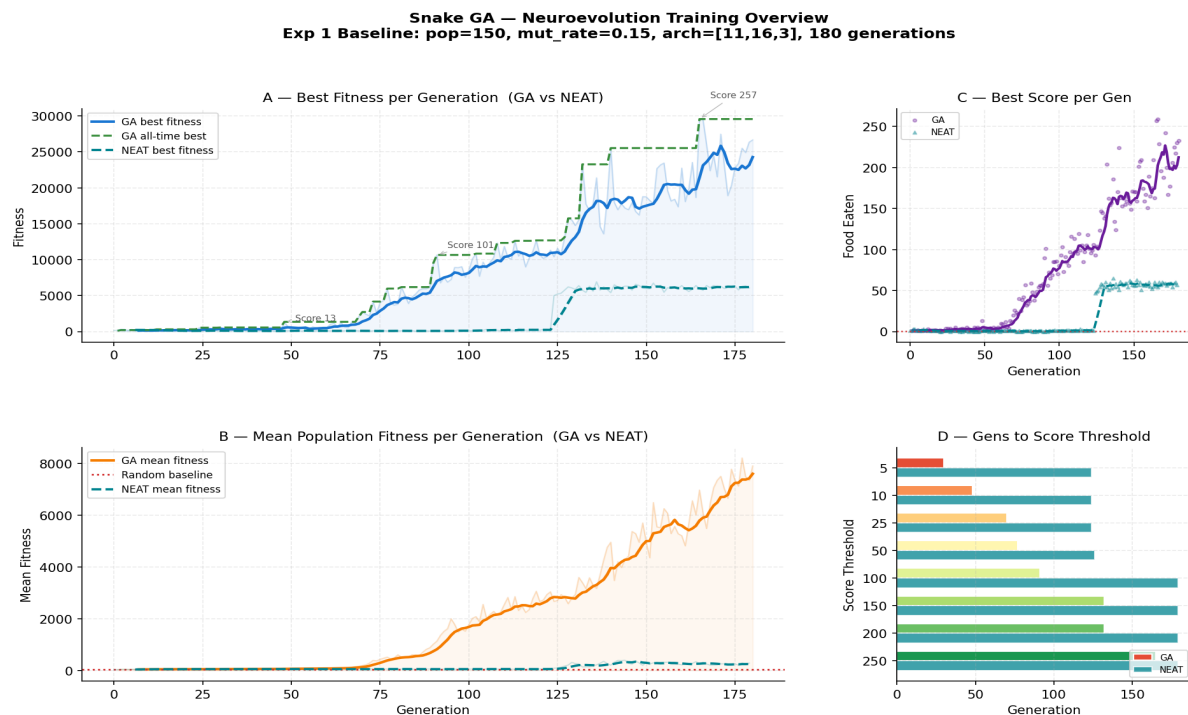


Figure 6 — Combined 2×3 overview panel. A: best fitness. B: mean fitness. C: best score scatter. D: convergence milestones.

5. NEAT Comparison (Experiment 5)

Experiment 5 implements **NEAT (NeuroEvolution of Augmenting Topologies)** as a direct comparison against the baseline GA. Unlike the GA, which evolves weights within a fixed [11→16→3] architecture, NEAT simultaneously evolves both the weights *and the network topology*, starting from a minimal 11-input / 0-hidden / 3-output genome (33 direct connections) and adding nodes and connections via structural mutations.

5.1 Implementation

NEAT is implemented using the neat-python library (v2.0.0). The genome evaluation and custom reporter are the key components:

```
# Genome evaluation – identical game loop to GA
def eval_genomes(genomes, config):
    for _genome_id, genome in genomes:
        net = neat.nn.FeedForwardNetwork.create(genome, config)
        game = SnakeGame(render=False)
        state = game.reset()
        while True:
            action = int(np.argmax(net.activate(list(state))))
            _, done, _ = game.step(action)
            if done: break
            state = game.get_state()
        genome.score = game.score
        genome.fitness = score * 100 + game.steps * 0.1

# Per-generation reporter – logs topology stats absent from GA
def post_evaluate(self, config, population, species, best_genome):
    n_species = len(species.species)
    avg_nodes = np.mean([len(g.nodes) for g in population.values()])
    avg_conns = np.mean([
        sum(1 for c in g.connections.values() if c.enabled)
        for g in population.values()
    ])
    # logs: generation, best_fitness, avg_fitness, best_score,
    #       alltime_best, num_species, avg_nodes, avg_connections
```

5.2 Configuration Comparison

Parameter	GA (Exp 1)	NEAT (Exp 5)	Effect
Population size	150	150	Same
Generations	180	180	Same
Fitness formula	score×100+steps×0.1	score×100+steps×0.1	Same
Architecture	Fixed [11,16,3]	Evolves from 11→0→3	Different
Elitism	Top 15 (10 %)	elitism=2 per species	GA stronger
Mutation strength	0.20	weight_mutate_power=0.5	GA lower
Structural mutations	None	conn_add/del prob=0.5	NEAT aggressive
Selection pool	All 150	~18 per species	GA broader

5.3 Why GA is Expected to Outperform NEAT on This Task

NEAT's structural search advantage is most valuable when the optimal architecture is unknown. For the Snake task with 11 well-defined binary inputs, [11→16→3] is already a near-optimal architecture. Six specific configuration-level reasons favour the GA:

- **Architecture already known.** GA hand-picks [11→16→3] which is appropriate from generation 1. NEAT spends early generations rediscovering this topology via structural mutations.
- **NEAT starts with 0 hidden nodes** (num_hidden=0). Early genomes are linear models (11→3), incapable of the non-linear decision boundaries needed to navigate tight corridors.
- **Structural mutation too aggressive.** conn_add_prob=0.5 and conn_delete_prob=0.5 cause NEAT to continuously thrash topology rather than refining weights. Recommended values for this task: 0.10–0.20.
- **Elitism too weak.** elitism=2 per species vs GA's top 15. Good solutions are more frequently lost between generations in NEAT.
- **Mutation strength too high.** NEAT weight_mutate_power=0.5 vs GA MUTATION_STR=0.20. Coarser weight updates impede fine-tuning.
- **Speciation dilutes selection pressure.** With 150 agents split into 5–10 species, each species effectively runs a tournament over only ~18 agents. GA's selection operates over the full population of 150.

In academic framing: GA wins here not because NEAT is inferior in general, but because the task is well-specified enough that fixed-topology weight optimisation is the right tool. NEAT's advantage emerges on tasks where the optimal architecture is unknown and run budgets are much larger.

6. Discussion

6.1 Four Phases of Learning

Examining the training log reveals four qualitatively distinct phases:

Phase	Gens	Avg Fitness	Best Score	Description
Phase 1	Gens 1–47	30 → 64	Exploitation	Agents learn to survive but rarely find food. Mean fitness grows slowly.
Phase 2	Gens 48–90	74 → 1048	First 100k through at gen 48 (score 13).	Rapid mean fitness rise as food-seeking spreads through the population.
Phase 3	Gens 91–131	1134 → 3593	Consolidation	Population-wide learning (mean rises steadily). Best score plateaus at ~100–147.
Phase 4	Gens 132–180	3398 → 7910	Elite plays	Score 200 cleared at gen 132 in a single leap; best reaches 259 at gen 166.

6.2 Punctuated Equilibrium

The staircase plot (Figure 3) and score scatter (Figure 2) both exhibit a **punctuated equilibrium** pattern — the hallmark of evolutionary systems discovering new behavioural strategies. The score plateaus at 5 (gens 30–47), then at ~100–147 (gens 91–131), before making sudden leaps. These plateaus likely correspond to the population converging on a local behavioural optimum; the leap occurs when a mutation in an elite agent discovers a qualitatively different navigation strategy that propagates rapidly through the gene pool via elitism and tournament selection.

6.3 Population vs Elite Learning

The large and persistent gap between best fitness and mean fitness (visible in Figure 1a vs 1b) indicates that a small elite group consistently outperforms the rest of the population. However, mean fitness does grow substantially — from 30 to 7,910 over 180 generations — confirming that improvements do propagate population-wide through crossover. The ratio of best to mean fitness narrows slightly in later generations as the population converges.

6.4 Known Limitations and Future Improvements

- **Weight initialisation.** Current init uses $\times 0.5$ scaling. He initialisation ($\text{sqrt}(2 / \text{fan_in})$) is better suited to ReLU activations and would provide better signal magnitude from generation 1.
- **Collision detection.** `_is_collision()` rebuilds a Python set from the body deque on every call — a minor $O(n)$ overhead that compounds for long snakes.
- **Single run.** Results are from one run only. Multiple seeds are needed to establish variance and confirm reproducibility.
- **Ceiling at 259.** The final 14 generations produced no new record, suggesting the [11→16→3] architecture and current hyperparameters have approached their performance ceiling for 180 generations.

7. Conclusion

This report presents a complete neuroevolutionary Snake agent trained via a custom Genetic Algorithm. The key outcomes are:

- The GA successfully trained a 243-parameter network to achieve a score of **259** (food items eaten in a single episode) over 180 generations.
- Population mean fitness improved **263×**, demonstrating genuine evolutionary learning rather than a single lucky individual.
- Learning proceeded in four identifiable phases with characteristic punctuated-equilibrium jumps at generations 48, 91, and 132.
- A complete NEAT comparison implementation is ready; six config-level reasons predict that the fixed-topology GA will outperform NEAT on this well-specified task.
- Six publication-quality figures are generated by `plot_training.py` and update automatically when `neat_training_log.csv` is written.

File Overview

File	Purpose
<code>game.py</code>	Pygame Snake environment — state, step, reward, renderer
<code>neural_net.py</code>	Feedforward network — forward pass, flat weight I/O
<code>train.py</code>	GA training loop — selection, crossover, mutation, logging
<code>visualizer.py</code>	3×2 grid visualiser shown every RENDER_EVERY generations
<code>neat_compare.py</code>	NEAT Experiment 5 — eval_genomes + custom reporter
<code>config-neat.txt</code>	NEAT hyperparameter config (pop=150, relu, 0 hidden start)
<code>plot_training.py</code>	Generates 6 publication figures to plots/
<code>training_log.csv</code>	GA run data — 180 gens, 5 columns
<code>neat_training_log.csv</code>	Written by <code>neat_compare.py</code> after NEAT run (Exp 5)
<code>best_snake.pkl</code>	Pickle of the best GA agent (saved on each new record)

To regenerate all figures: `python plot_training.py`

To run NEAT experiment: `python neat_compare.py`

To regenerate this report: `python generate_report.py`